

Designing camps for students who've never coded

Five days, fifty students, zero programming experience. What we kept changing until it worked.

G. Zhang · 7 min read · September 2025

Every summer we run a week-long camp for students who have never written a line of code. Fifty of them, ages eight to twelve, five days, six hours a day. The first time we did this, in collaboration with a partner organization, it was a disaster in slow motion. The students were happy. The parents were happy. The exit surveys were full of smiling faces. And when we followed up with the same kids three months later, almost none of them had written any code, started any project, or remembered the names of the concepts we had spent the week on. We had successfully run a babysitting service with robots in it.

This essay is about what we changed, season by season, until we had a camp that actually produced students who left as beginners rather than as spectators. Some of the changes are obvious in retrospect. None of them were obvious at the time.

The myth of the gentle on-ramp

The first instinct, when designing a beginner camp, is to start with the gentlest possible introduction. Drag-and-drop blocks. Cartoon characters. Pre-built robots that already work, so the student can experience success on day one. We did all of this in our first camp. We thought we were being good educators. What we were actually doing was setting up a five-day arc in which the difficulty curve was almost flat for the first three days and then spiked sharply when we tried to introduce anything real, like a conditional or a variable, in the second half of the week.

Students who have just spent two days dragging a green flag block to make a cat say hello do not have a foundation for what a variable is. They have a foundation for being entertained. The gentle on-ramp does not build up to the harder material. It delays it, and the delay makes the harder material harder when it eventually arrives, because the student has been taught for two days that programming is something that happens through

dragging colorful blocks around, and now you are asking them to think about something that does not look like that at all.

What works better, we eventually figured out, is to introduce the hard idea on day one, in the smallest possible form, with no pretense that it is easy. The first thing we teach now, in the first two hours of the first day, is what a variable is. Not how to use one. What one is. We do this with physical objects, not code. Every student gets a paper cup and a stack of tokens. The cup is the variable. The tokens are the value. You can put tokens in. You can take tokens out. You can check how many are in there. You can compare what is in your cup to what is in someone else's cup. The cup has a name written on the side. We spend two hours doing nothing but this, with progressively more elaborate games built around it.

By the end of those two hours, every kid in the room knows what a variable is, in a way that they will not lose. They have moved the tokens with their own hands. They have argued with their neighbor about whose cup has more in it. They have figured out, with prompting, that two cups with the same name are confusing and you should probably not do that. We have not written a single line of code. We are not going to write code until tomorrow. The cups are the foundation, and the cups are doing more work than the first two days of any block-based curriculum we ever tried.

This pattern, introduce the abstract idea through a physical analog, then write code that matches the analog, became the spine of our entire camp. Loops are introduced with a circle of chairs you walk around. Conditionals are introduced with a game where one student asks a yes-or-no question and the other student does one of two things based on the answer. Functions are introduced with the idea of a recipe card, which the student writes once and then hands to another student who follows it. By the time the code shows up, the kid already knows what the code is doing, because they did it with their body first.

What "no coding experience" actually means

The second thing we got wrong, in our first camp, was assuming that "no coding experience" meant a homogeneous group. It does not. It means a group with wildly different starting points who happen to share one specific gap.

Some of the students who have never coded have never used a computer in any extended way. They do not know what a file is. They do not know what double-click means. They can use a tablet but get lost on a laptop. Some of the students who have never coded are eleven-year-olds who have been on a screen for six hours a day for years and have a sophisticated mental model of how applications work, they just have never written one. Some have done Minecraft Redstone, which is functionally a programming environment,

and have intuitions about logic gates that they cannot articulate but absolutely have. Some are recent immigrants who can read English but for whom English is the second or third language and following spoken instructions in a noisy room is the actual bottleneck, not the programming.

Treating these students as the same group, because they share the label "beginner," means the camp lands well for the middle of the distribution and badly for both tails. The kids on the high end are bored by lunch on day one. The kids on the low end are lost by lunch on day two. The bored kids start disrupting. The lost kids go quiet. By the middle of day three, your camp has bifurcated into two groups that need different things, and you are running one curriculum.

What we now do is a brief assessment on the first morning, before the official content begins. Not a test. A series of small tasks, framed as games, that let us watch each student do something. Can they type their name on a keyboard? Can they follow a two-step instruction without re-asking? Can they recognize that two pictures of robots are showing the same thing in different orientations? We watch and tag, in about thirty minutes, where on a few axes each student falls. We then assign them to tables that group similar students together, not by age but by where they actually are. The eight-year-old who has been on a screen forever and the eleven-year-old who has barely touched a keyboard might sit at different tables. The kid's friend they came with is in a different group. This is uncomfortable for the first hour. By the end of day one, it is invisible, because each table is working at a pace that matches its students.

The thing that has to happen on day three

The most important moment in a five-day camp is not the demo on day five. It is the moment, on day three, when each student writes their first piece of original code. By original I mean code that solves a problem the student chose, not a problem we assigned. By that point in the week they have learned enough syntax to write something simple. The question is what they choose to write.

We used to skip this. We had a curriculum that ran straight through the week with assigned projects, and the kids did them, and they were fine, and nothing of theirs ended up in any of it. We changed this after we noticed that the students who, on their own, started modifying our assigned projects in small ways were dramatically more engaged than the students who completed the assignments as written. They were the kids who had taken some ownership over what they were doing, and the assignments had not given them anywhere to put that ownership.

Day three now has a specific block, two hours long, called What Do You Want To Make. The mentors do not propose projects. They ask. The student is required to come up with something. Anything. If they cannot think of anything, the mentor asks them what they wish their robot could do, and writes down the answer, and helps them break it into the smallest possible version of that thing. A kid who said they want their robot to be a dog spends the rest of the afternoon making it bark when a button is pressed. A kid who said they want it to find the door spends the afternoon making it turn when a sensor sees something dark. The projects are small. They are absurd, often. They are entirely the student's, and that is the only thing that matters.

The fifth day's demos are then built around these projects, not around the assigned ones. A parent comes in and watches their kid show a robot that barks. The kid is not embarrassed by how small it is. The kid is proud, because they made the robot bark, and they can explain to their parent how the button works, and three months later when we follow up they remember this. They do not remember the assigned line follower from day two. They remember the dog they made on day three.

What we stopped doing

We stopped trying to teach everything. The original camp had a syllabus that tried to cover variables, loops, conditionals, functions, sensors, motor control, basic algorithms, and an introduction to autonomous behavior. In five days. For students who had never coded. This was, in retrospect, an act of pedagogical malpractice that I am embarrassed we attempted.

We now cover three things. Variables, conditionals, and how to use one sensor to make something happen. That is it. Loops get a passing mention if a kid asks. Functions do not get covered at all. We could cover them. The students could probably handle them in a basic form. But the marginal hour spent on functions is an hour not spent reinforcing variables, and the choice between teaching one more thing badly or teaching three things well is not actually a hard choice once you let go of the idea that the syllabus has to be impressive.

The parents do not check the syllabus. The parents check whether their kid is happy and whether they can show them something at the end. The students do not check the syllabus either. They check whether they made something. Pretending that we are teaching computer science in five days to an eight-year-old is a kind of vanity. We are introducing them to the idea that programming exists, that they can do it, and that doing it is fun. That is the entire goal. Everything in the camp now serves that goal, and everything that does not has been cut.

What we tell partner organizations

When other organizations ask us to help them design a beginner camp, the advice we give them is the same every time. Pick fewer concepts than you think you need. Introduce each one physically before you introduce it on the screen. Group students by where they actually are, not by what age they are. Build a day three where the students choose what they make. End with a demo where the parent watches their own kid's project, not a polished group performance. Follow up three months later and ask what they remember.

The follow-up is the part most programs skip. We skipped it for two years. The follow-up is the only honest measure of whether your camp did anything, because the smiling faces at the end of day five are not a measure of anything except that the week was fun. Fun is necessary but not sufficient. The question is what the kid does in October, when no one is asking, when there is no instructor in the room. The students who, on their own, open a tablet and try to remember how to make the cat move, those are the students the camp worked for. We try to make as many of those as we can, and we have learned to be honest with ourselves about how many we are actually making, which is fewer than we wish but more than we used to.

Growing Up with Robotics runs week-long summer camps in partnership with community organizations across the DMV region. Camp design materials are available to partner programs.